Open Access | https://doi.org/10.25080/mhpx3742

Toward Reliable Localization of Free and Open Source Software: LLM-assisted Translation Workflows for QGIS

Yu-Lin Chen[1,2], **Tyng-Ruey Chuang**[2,3,4], and **Cheng-Jen Lee**[2]

[1]National Sun Yat-sen University, Taiwan, [2]Institute of Information Science, Academia Sinica, Taiwan, [3]Research Center for Humanities and Social Sciences, Academia Sinica, Taiwan, [4]Research Center for Information Technology Innovation, Academia Sinica, Taiwan

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference

ISSN: 2575-9752

Abstract

Localization for free and open source software requires more than fluent target-language text. In Qt `.ts` files, translations must remain valid software resources: runtime placeholders, markup, escaped entities, line breaks, plural forms, terminology, XML structure, and message metadata must be preserved or handled according to explicit rules. This paper presents a Python workflow that uses LLMs to support localization of Qt Translation Source (`.ts`) files, using QGIS Traditional Chinese localization as a case study. A Python script segments each source string into translatable human-language spans and protected format-control tokens such as placeholders, markup, entities, and line breaks. The LLM translates only the human-language spans, while Python reinserts the protected tokens and validates the reassembled result. The workflow supports both cloud-based and local LLMs.

Experiments based on several translation strategies on a fixed stratified subset of 3,000 source messages, as well as complete-corpus experiments on all 28,924 QGIS source messages, demonstrate trade-offs in LLM-assisted translation automation. Direct-generation settings that send complete source strings to the language models, including variants with glossary hints and multiple generated candidates, can achieve lower MQM error rates but still suffer from non-zero rates of structural failure. In contrast, our production workflow —masked template translation with glossary hints, top-three generated candidates per message, strict validation, and source-preserving fallback—produces final `.ts` artifacts with zero observed structural failures on the complete corpus for all three evaluated LLM backends: Grok, Gemini, and TAIDE. Fallback rows remain explicitly queued for human review rather than being treated as completed localization. This work therefore contributes not an LLM leaderboard or a method to improve linguistic quality of translation, but a reproducible workflow for making software localization with LLMs safer and auditable.

Keywords QGIS, localization, large language models, validation, Qt Linguist, open source software, translation memory, terminology management, machine translation, software engineering, Traditional Chinese, geographic information systems, reproducible workflows

Published Jul 13, 2026

Correspondence to
Yu-Lin Chen
aswendy062666@gmail.com

Open Access

Copyright © 2026 Chen *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Localization quality is a practical barrier to the adoption of scientific software. A localized interface must do more than translate the message strings: it must use clear language, keep terms consistent, and preserve software formatting rules. Taking QGIS, a widely used free and open-source geographic information system [1], [2], as an example, untranslated residual English strings, inconsistent GIS terms, and broken placeholders or markup can make the localized interface harder to use and reduce users' trust in the software.

LLMs can assist localization, but software localization is not merely document translation. A Qt `.ts` file is an XML-based localization resource containing source strings, translations, contexts, locations, message states, plural forms, and translator metadata. User-visible strings may contain runtime placeholders such as `%1` and `%n`, brace placeholders such as `{0}` or `{feature_id}`, Qt mnemonic markers for Alt-key shortcuts such as `&File`, HTML-like markup, escaped entities, numeric tokens, and significant whitespace. A translation that looks acceptable in plain-text review may still be release-blocking if it violates format constraints—for example, by omitting `%1`, altering a `<b>...</b>` pair, removing a required line break, or using an incorrect GIS term in the target language.

This paper presents a validation-first workflow for QGIS localization with LLM support, where generated translations are not trusted simply because they are language model outputs. In the masked production path, a generated translation passes the automated acceptance stage only if the reassembled `.ts` message is well-formed XML and has no emitted validation issue; accepted outputs and review-required fallbacks are distinguished in the external run log for audit and subsequent human review. The central claim of our work is not that LLMs will replace human translators. Rather, we see LLMs being placed inside a reviewable Python workflow to help software localization efforts where rule-based validation protects software structural integrity, project glossaries provide terminology guidance, LLM judges scale up linguistic quality review, and human reviewers oversee the entire process and retain final responsibility.

The workflow is intentionally not tied to one LLM backend. Translation can be performed through cloud models or local models. This matters for FOSS communities because contributors may prefer local inference for cost, privacy, offline processing, or reproducibility, while others may use cloud models for higher translation quality or better support for JSON-style structured responses. The paper therefore evaluates our proposed workflow across several models.

Contributions. This paper makes four contributions:

1. A compact validation-first Python workflow for QGIS `.ts` localization with LLM support.
2. A structure-preserving template translation method in which LLMs translate only human-language spans while Python scripts preserve and protect format-control tokens and reassemble the translated messages by rules.
3. A shared backend interface for both cloud-based and local LLMs, enabling the same workflow to run, for example, with Gemini, Grok, and a local TAIDE-family model.
4. A reproducible evaluation protocol using fixed subsets, rule-based structural checks, terminology diagnostics, production runs on the complete corpus, as well as a sampled evaluation of semantic quality of the end results using MQM.

2. Background and Related Work

2.1. Software localization as structured data transformation

The Qt Linguist TS format (`.ts`) is not merely a plain-text format for natural-language translation. Instead, it is an XML-based file format described by an XML Schema Definition (XSD), which specifies the permitted structure and elements of the XML document, and its message records may contain message type, source text, translation text, context, locations, comments, and plural forms [3]. This makes `.ts` localization closer to a transformation of structured data with explicit constraints than to unconstrained natural language translation.

In this setting, a localization workflow should distinguish between:

```
Human-language content: words and phrases intended for translation.
Formatting-language content: string tokens that must be preserved exactly.
```

A generic LLM prompt that treats the source as plain text may produce readable translations while silently damaging format-control tokens. This motivates a translation workflow that is aware of protected tokens, rule-based validation, and explicit reporting. Localization engineering and computer-assisted translation literature similarly treats localization as a managed workflow involving software resources, terminology, translation consistency, testing, and quality assurance (QA) rather than a single act of sentence translation [4], [5]. This paper follows that workflow-oriented view, but makes the structural QA layer executable and reproducible in Python.

2.2. QGIS terminology and Traditional Chinese localization

QGIS localization is domain-specific. Terms such as `layer`, `feature`, `raster`, `vector`, `CRS`, `geometry`, `attribute table`, and `processing algorithm` must be handled consistently after translation (in our case, into Traditional Chinese) across their occurrences in menus, processing tools, database modules, and error messages. Generic LLM translation may produce fluent output while terms drift across terminology variants.

To reduce this drift, the workflow uses a glossary-assisted retrieval approach, in which relevant entries from a GIS glossary are retrieved as reference hints for generation. Glossary entries are not used as mechanical replacement rules because GIS terms often appear in derived or compound forms. For example, `raster`, `raster layer`, `rasterize`, and `rasterization` may require different Traditional Chinese expressions. Therefore, the workflow retrieves glossary hints, includes them in the prompt, and validates output after generation rather than enforcing direct string substitution.

2.3. Structured generation, token preservation, and semantic quality evaluation

Methods for structured output can reduce malformed model responses by asking the model to return a machine-readable object such as JSON [6], [7]. In software localization, however, valid JSON is not enough: placeholders, markup, Qt mnemonic markers, and whitespace still need rule-based checks. In the masked conditions, this workflow therefore uses structured JSON output only for translated text spans. Protected tokens are not generated by the model; they are reinserted by Python code. In this paper, the phrase structured generation refers to a JSON text-span response, while token preservation refers to a separate rule-based layer.

Semantic quality is evaluated separately. MQM is an analytic translation quality evaluation framework applicable to human translation, machine translation, and AI-generated translation [8], [9]. T. Kocmi and C. Federmann [10] introduced GEMBA-MQM, an LLM-based method for detecting translation quality error spans using a fixed few-shot prompt and an MQM-style rubric. In this paper, LLM-as-a-judge is used as scalable assisted annotation, not as the ground truth. Model and condition labels are hidden from the judge, and rule-based structural results are reported separately from semantic MQM scores.

3. Method

3.1. Workflow overview and LLM backends

The selected masked workflow has six stages; the experimental conditions are defined separately below:

1. Parse the Qt `.ts` file into translation units.
2. Extract mnemonic metadata and split each source string into protected tokens and translatable text spans.
3. Retrieve glossary hints from tabular terminology resources for the source segment.
4. Ask an interchangeable LLM backend to translate only the text spans.
5. Reinsert protected tokens, restore mnemonic metadata, validate candidates, and use a source-preserving fallback when needed.
6. Validate the generated `.ts` file and emit structural, rule-based QA, and MQM reports.

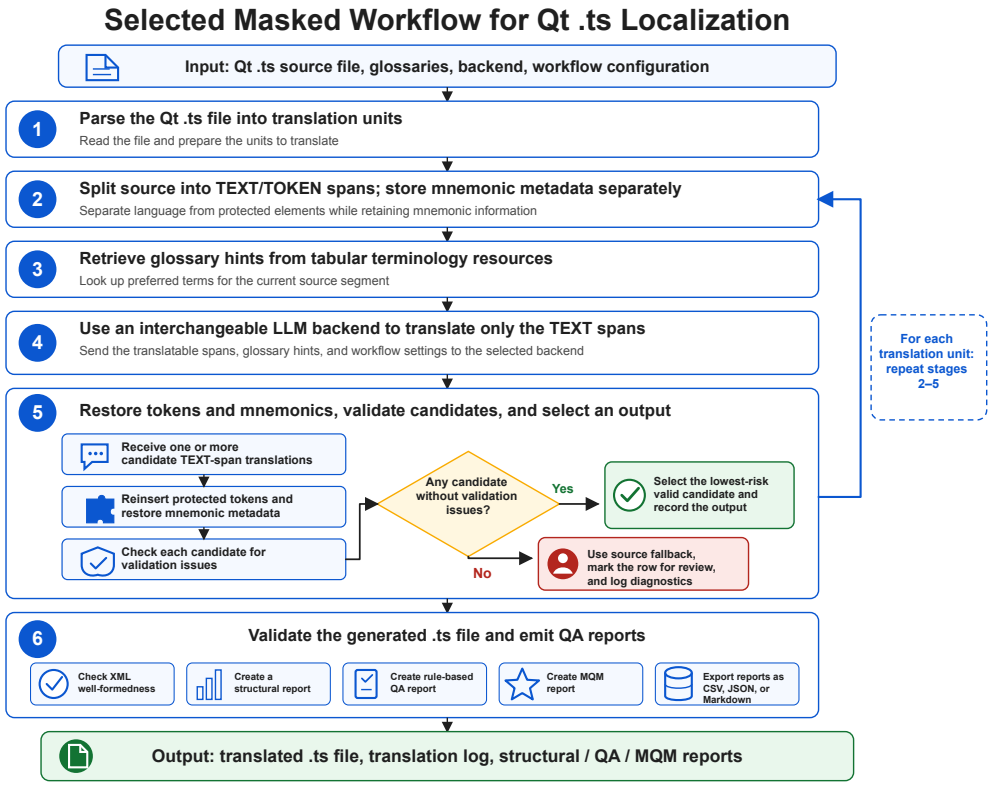


Figure 1. *Workflow for Qt `.ts` localization. Stages 2-5 are repeated for each translation unit; protected tokens and mnemonic metadata are restored before a candidate is selected or a source-preserving fallback is recorded for review.*

Figure 1 separates rule-based processing from model-dependent generation. The tasks of parsing, token splitting, glossary lookup, token reinsertion, candidate selection, and validation are implemented by Python scripts and are shared across all backends. The backend adapter is the only part that changes when the workflow invokes different LLMs, for example by calling Gemini, Grok, or a local TAIDE model. This design makes it possible to evaluate whether the same workflow remains reliable across different models, rather than focusing on specific model providers.

The rule-based and glossary layers are applied uniformly across all model backends. The only backend-specific component is the adapter that converts a common request object into a cloud API call or a local inference call.

Table 1. *Model backends.*

Backend	Access	Model ID	Role
Gemini	Cloud	`gemini-3.1-flash-lite`	Translation backend

Grok	Cloud	`grok-4.3`	Translation backend and MQM judge
TAIDE	Local	`taide/Gemma-3-TAIDE-12b-Chat-2602`	Local translation backend

Model version details are kept out of Table 1 to reduce space. Gemini and Grok are cloud backends with support for structured outputs [6], [7], [11], [12]. The TAIDE backend is a Hugging Face Hub checkpoint whose model card states that it is based on `google/gemma-3-12b-pt`; the base Gemma 3 12B pre-trained checkpoint and Gemma 3 technical report provide the relevant Gemma 3 model context [13], [14], [15]. Because cloud aliases and model repositories can change over time, the reproducibility artifact described in Reproducibility and Artifact Availability records the model identifiers, condition parameters, sampling seed, prompt information, available runtime defaults, and source-file checksum. Historical details that were not preserved in the supplied run records, such as provider-side request timestamps, the exact local checkpoint revision, package versions, and GPU model, are explicitly marked as not recorded rather than inferred.

3.2. A model of translation units and token sequences

Each Qt message is represented as a translation unit:

$$u = (id, i, c, s, t, m, l, a), \tag{1}$$

where id is a stable identifier, i is the message index, c is the Qt context, s is the source string, t is the target translation, m contains message metadata, l contains source-code locations, and a contains extracted artifacts such as placeholders, tags, entities, numeric tokens, accelerator markers, and newline tokens.

For a source string s, let $M(s)$ denote extracted Qt mnemonic metadata, such as `&A`. Because the mnemonic key character is also part of the human-readable source text, mnemonic markers are represented separately rather than as literal protected-token spans. Let s' denote the source with only the mnemonic ampersand marker removed, while retaining the mnemonic character itself. The splitter then produces an alternating sequence of text spans and protected tokens:

$$s' = x_0\tau_1 x_1\tau_2 \cdots \tau_k x_k, \tag{2}$$

where x_j are translatable or copyable text spans and τ_j are protected formatting-language tokens. The protected token sequence is:

$$T(s') = (\tau_1, \tau_2, ..., \tau_k). \tag{3}$$

For any accepted output $\hat{t}$, the multiplicities of literal tokens and mnemonic markers are checked separately:

$$\mathrm{Counter}(T_r(s')) = \mathrm{Counter}\big(T_r(\hat{t})\big) \quad \forall r \in \mathcal{R}, \qquad \mathrm{Counter}(M(s)) = \mathrm{Counter}\big(M(\hat{t})\big), \tag{4}$$

where $\mathcal{R}$ is the set of token extractors, such as Qt placeholders, brace placeholders, printf placeholders, HTML/XML tags, entities, numeric tokens, and newlines. The validator checks token multiplicities, while masked reassembly preserves their template positions. Separating mnemonic metadata allows an English in-word mnemonic such as `&A` to be represented in the Traditional Chinese translation as the suffix `(&A)`.

As an example, consider the source message that motivated this distinction:

```
<message>
  <location filename="src/app/qgsattributedialog.cpp" line="125"/>
  <source>Open &amp;Attribute Table</source>
  <translation type="unfinished"></translation>
</message>
```

The corresponding translation unit can be represented as:

```
u = (
  id = "QgsAttributeDialog:125",
  i  = 125,
  c  = "QgsAttributeDialog",
  s  = "Open &Attribute Table",
  t  = "",
  m  = {translation_type: "unfinished"},
  l  = [("src/app/qgsattributedialog.cpp", 125)],
  a  = {
        qt_mnemonic_markers: ["&A"]
       }
)
```

Before splitting, the mnemonic is extracted and its ampersand is removed from the source text:

```
M(s)    = ("&A",)
s_prime = "Open Attribute Table"
```

The splitter then produces this alternating sequence for s':

```
T0 = "Open Attribute Table"
```

The protected token sequence T is therefore:

```
T(s_prime) = ()
```

The model is asked to translate only the text spans and may return JSON such as:

```
{
  "T0": "開啟屬性表"
}
```

Python code then restores the extracted mnemonic as a suffix:

```
開啟屬性表(&A)
```

The parsed string uses `(&A)`; in `.ts` XML it is serialized as `(&A)`. At runtime, Qt displays `(A)`, with `A` acting as the mnemonic key.

The accepted output preserves the checked token sequence and mnemonic separately:

```
T(s_prime) = T(accepted output) = ()
M(s)       = M(accepted output) = ("&A",)
```

This example also shows the boundary of the model's responsibility. The model chooses the Chinese terms for `Open` and `Attribute Table`, but it does not need to regenerate the accelerator marker. The protected-token example below illustrates the same separation for markup and placeholders.

3.3. Structure-preserving masked template translation

After mnemonic extraction, each source string is split into translatable `TEXT` spans and protected tokens. This separation is referred to as masking in the experimental conditions. Protected tokens include:

```
Qt placeholders:                %1, %2, %n, %L1
Brace placeholders:             {0}, {feature_id}
printf placeholders:            %s, %d, %.2f
HTML/XML tags:                  <b>, </b>, <p align="right">
Escaped entities:               &amp;, &lt;, &gt;,  
Escaped controls:               \n, \t, \r
Actual controls:                newline, tab, carriage return
Literal ampersand escape in Qt: &&
Numeric literals:               100, 3.44, -1
```

Qt mnemonic annotations are handled separately as accelerator metadata, allowing an in-word marker such as `&A` in `&Attribute` to be represented as `(&A)` in Traditional Chinese. The LLM is not used to reproduce either protected tokens or mnemonic markers. It translates only the human-language spans and returns JSON keyed by text-span identifiers. The Python script then restores the protected tokens and mnemonic markers by rule.

Example source:

```
<p align="right">Algorithm author: {0}</p>
```

Template spans:

```
TOKEN: <p align="right">
TEXT : Algorithm author:
TOKEN: {0}
TOKEN: </p>
```

Model response:

```
{"T0": "演算法作者： "}
```

Reassembled output:

```
<p align="right">演算法作者： {0}</p>
```

This design makes preservation of checked token classes rule-based for accepted candidate translations. The workflow can preserve the defined format rules, but it cannot guarantee that the translation has the right meaning, sounds natural, or otherwise reads well in the user interface.

3.4. Glossary retrieval from tabular resources

Translation project glossaries are provided as spreadsheet-style terminology tables. Our method only assumes that each table can be normalized into records containing:

```
source_term, target_terms, forbidden_terms, priority, domain, note
```

Here, `forbidden_terms` are target-language expressions that the project glossary marks as discouraged or disallowed for a given source term.

To avoid scanning every glossary entry when translating each message segment, source terms are tokenized, normalized, and indexed by n-gram keys:

$$I[n, g] = \{e \in G : \text{key}_n(e.source_term) = g\}, \tag{5}$$

where G is the glossary and g is a normalized n-gram key. For a source segment s, the retrieved hint set is:

$$H(s) = \text{TopK}\left(\bigcup_{n=1}^{N_{max}} \bigcup_{g \in \text{ngrams}_n(s)} I[n, g], K\right). \tag{6}$$

Here, $H(s)$ denotes the glossary hints retrieved for segment s, N_{max} is the maximum n-gram length considered, and K is the maximum number of hints passed to the model. In the implementation, matched entries are sorted by source-term length so that more specific multi-word terms are preferred over shorter terms.

Glossary hints are passed to the LLM as contextual guidance, not mechanical replacements. This allows the model to adapt terminology to context while keeping prompt size controllable.

3.5. Candidate selection

Each segment may produce multiple candidate translations. The default production condition generates three candidates. A Python script scores each candidate with a rule-based quality-risk function:

$$Q(\hat{t}) = \sum_{e \in E(\hat{t})} w_Q(severity(e)) + \lambda_1 R_{latin}(\hat{t}) + \lambda_2 R_{len}(s, \hat{t}). \tag{7}$$

Here, $\hat{t}$ is a candidate translation for source segment s, $E(\hat{t})$ is the set of validation issues detected in the candidate, and w_Q maps critical, major, minor, and informational issues to 100, 10, 1, and 0, respectively. $R_{latin}(\hat{t})$ penalizes unexpected Latin-script residue, $R_{len}(s, \hat{t})$ penalizes extreme source-target length ratios, and the fixed weights are $\lambda_1 = 2$ and $\lambda_2 = 3$.

For the masked production conditions, the accepted candidate is selected only from candidates with no validation issues:

$$j^* = \arg \min_{j : E(\hat{t}_{u,j}) = \emptyset} Q(\hat{t}_{u,j}). \tag{8}$$

Here, $\hat{t}_{u,j}$ is the j-th candidate for translation unit u. In this strict masked path, every emitted validation issue blocks automatic acceptance.

If no generated candidate passes strict validation in a masked condition, the source string is written as a source-preserving fallback and the translation unit is logged as review-required; such a fallback is not counted as completed localization. In the current implementation, this status is recorded in the external run log rather than in the `.ts` `unfinished` attribute, so the log must accompany the generated artifact during review. In conditions C0 and C2, the selected raw candidate and its detected issues are retained so that structural failures remain observable. Review state is not used as a key quality metric

because it may have several operational causes: no valid candidate, backend exception, malformed JSON, or intentionally conservative rejection. Detailed review-state counts are reported in the accompanying reproducibility artifact. This paper focuses on structural failure rate, deterministic rule-based QA diagnostics, and MQM semantic quality.

3.6. Guarantee boundary of translation validation

The guarantee boundary is intentionally narrow:

- The workflow preserves many, but not all, token classes for accepted masked-template outputs.
- The evaluated masked-condition bundles yielded zero observed structural failures under the implemented extractors.
- It does not prove semantic adequacy, fluency, terminology correctness, UI usability, or cultural appropriateness.

If a token class is not defined by the extractor, it is outside the guarantee boundary. If a segment is logged for review rather than accepted as a structurally valid candidate, it should not be counted as completed user-facing localization. If a result has zero observed structure failures on a finite subset or corpus, this should be reported as zero observed failures under the checked token classes, not as proof of a 0% population error rate.

3.7. Workflow procedure

The procedure is summarized below in pseudocode. Full implementation details are in the code repository.

```
Input: Qt TS file S, glossaries G, backend b, condition C
Output: translated TS file, translation log, rule-based reports

U <- parse_ts(S)
I <- build_glossary_index(G)

for each translation unit u in U:
    if C.masking:
        s_prime, M <- extract_mnemonic_metadata(u.source)
        X, T       <- split_text_and_tokens(s_prime)
    else:
        X <- [u.source]; T <- (); M <- ()
    H    <- retrieve_glossary_hints(I, u.source, top_k)
    Y    <- generate_candidates(b, u, X, T, H, C)
    V    <- empty list

    for each candidate y in Y:
        if C.masking:
            t_hat <- restore_mnemonic(reinsert_tokens(y, T), M)
        else:
            t_hat <- y
        issues <- validate(u.source, t_hat)
        if not C.masking or no issue is found:
            add (t_hat, rule-based_risk_score(t_hat)) to V

    if V is not empty:
        write the lowest-risk candidate and log its issues
    else:
        write u.source as a source-preserving fallback and log review-required status

validate XML well-formedness
emit CSV, JSON, and Markdown reports
```

4. Implementation and Experimental Conditions

Our implementation is a Python command-line workflow. The main runner builds fixed subsets, runs the C0-C4 experimental conditions below, writes generated `.ts` files, and calls evaluation and scoring scripts. The same selected subset is reused across model backends to enable pairwise comparisons.

Table 2. *Experimental conditions.*

Condition	Masking	Glossary hints	Candidates	Purpose
C0 (Direct)	no	no	1	Direct translation without masking
C1 (Complete workflow)	yes	yes	3	Production condition
C2 (Direct + glossary)	no	yes	3	Tests glossary/candidates without masking
C3 (Masked without glossary)	yes	no	3	Tests masking without glossary hints
C4 (Single candidate)	yes	yes	1	Tests the value of generating multiple candidates

C0-C4 are experimental conditions for the same workflow over a fixed stratified subset of 3,000 message segments. C0 and C2 disable the structural hard lock and retain selected raw outputs so that direct-generation failures remain measurable. C1, C3, and C4 use masking, rule-based reassembly, strict validation, and a source-preserving fallback for rejected candidates. The conditions therefore compare complete operating configurations rather than isolating masking from the fallback policy. Our complete-corpus evaluation focuses on C1 because it is the selected production configuration, combining masking, glossary hints, and three candidates. Scaling up all experimental conditions would increase cost and review burden without being necessary to test whether this selected configuration produces structurally valid final artifacts for the QGIS Traditional Chinese localization workflow.

4.1. Direct generation and residual risk

A natural question is whether stronger prompting is sufficient. C0 and C2 should be interpreted conservatively: they demonstrate residual risk under the implemented direct prompts, but they are not an exhaustive benchmark of every possible prompt-only or constrained-decoding design. In these conditions without masking, the model sees the complete source string and remains responsible for reproducing protected formatting-language tokens. C2 additionally receives glossary hints and three candidates. These settings can improve semantic quality, but they still leave protected-token preservation to the model. Masked conditions change the problem decomposition: the LLM translates only human-language spans, while Python code preserves and reassembles protected tokens, validates the result, and substitutes the source when every candidate is rejected.

4.2. Validation fixtures and smoke tests

Our artifact includes a bundled mini evaluation fixture for exercising the rule-based layer. The fixture contains archived 100-segment C0-C4 `.ts` outputs, condition metadata, selected-segment metadata, and evaluation outputs. The default reproduction will rerun the rule-based evaluator and scoring scripts on these archived `.ts` files, so that token checks, accelerator handling, XML well-formedness checks, condition comparison, scoring behavior, MQM request planning, and table generation can be verified without calling any model API.

The fixture is not presented as a full regeneration of the paper experiments: it does not rerun LLM translation, candidate generation, glossary-assisted prompting, or token reassembly during generation. Those steps are covered by the workflow scripts and can be exercised through optional cloud/local LLM reruns, while the default reproduction path provides a stable offline check of the rule-based evaluation and reporting code path.

5. Evaluation

This section evaluates the effectiveness of our workflow. The purpose of the evaluation is not to rank the models we use. Rather, it is to validate the method we propose. It has two reported layers and one complementary semantic-review protocol.

1. **Subset condition comparison:** Grok, Gemini, and TAIDE are evaluated under conditions C0-C4 on the same 3,000-segment stratified subset of source messages.
2. **Complete corpus:** Grok, Gemini, and TAIDE are evaluated under C1 on all 28,924 QGIS source messages.
3. **Semantic review:** We use MQM-style sampled judging to evaluate semantic adequacy, terminology, fluency, and locale style on the C0-C4 outputs. This is separate from rule-based structural scoring.

The evaluation subset uses multi-label stratification based on source-string features that are likely to affect localization reliability, including plural/numerus messages, Qt placeholders, accelerator markers, HTML/XML content, glossary hits, numeric or code-like content, newline/control characters, long strings, and ordinary strings. Because these feature categories overlap, a segment may carry more than one label; the final subset is fixed and reused across all model backends and experimental conditions.

5.1. Structural metrics

For condition c and N checked segments, the structural failure rate is:

$$\hat{p}_c = \frac{1}{N}\sum_{u=1}^{N} 1[E_{struct}(u,c) \neq \emptyset]. \tag{9}$$

For a zero-failure sample, a one-sided approximate 95% upper bound can be summarized using the rule of three [16]:

$$p_{upper} \approx \frac{3}{N}. \tag{10}$$

For the 3,000-segment subset, this descriptive bound is approximately 0.1000%; because the subset is feature-stratified, the bound is not treated as a guarantee for a broader population. The complete 28,924-message corpus was enumerated rather than sampled, so its observed in-corpus result is reported directly as 0/28,924.

5.2. Deterministic rule-based QA metrics

Let E_{det} be the severity-weighted deterministic error points per 1,000 source characters:

$$E_{det} = 1000 \times \frac{\sum_u \sum_{e \in D_u} w_D(severity(e))}{\sum_u \mid s_u \mid}, \qquad RuleQA = \max(0, 100 - E_{det}), \tag{11}$$

where D_u contains deterministic structural, completion, untranslated-text, and terminology findings for segment u, with $w_D(critical) = 12$, $w_D(major) = 5$, $w_D(minor) = 1.5$, and $w_D(info) = 0$. Rule QA is therefore a dashboard diagnostic rather than a linguistic translation-quality score, and review-required status is reported separately. A condition can

improve structural safety while lowering Rule QA if it generates more untranslated or otherwise flagged results.

5.3. MQM-style semantic review metrics

Semantic quality is evaluated with MQM-style scoring. For segment u, let P_u be the set of MQM penalties assigned by the judge. The weighted MQM error rate is:

$$MQM\text{-}ER = 1000 \times \frac{\sum_{u=1}^{N} \sum_{p \in P_u} p}{\sum_{u=1}^{N} |\ s_u\ |}. \tag{12}$$

Lower values indicate fewer weighted semantic errors per 1,000 source characters. The fixed MQM penalties are 0, 1, 5, and 25 for neutral, minor, major, and critical findings, respectively. The auxiliary MQM score on a 0-100 scale is reported for readability; the error rate is the main semantic metric. For each backend-condition pair, Grok 4.3 judged three random samples of 200 segments, yielding 600 judgments. The same sampled segment positions were used across backends and conditions within each repeat. The reported approximate 95% half-width is $1.96s/\sqrt{3}$, where s is the standard deviation of the three run-level MQM-ER values; with only three repeats, this interval is descriptive rather than a precise population estimate.

The MQM analysis is not framed as a claim that masking should improve semantic quality. The trade-off question is whether the semantic cost of masking is small enough to justify the structural safety gain. Let:

$$\Delta_{MQM} = MQM\text{-}ER(C1_{masked}) - MQM\text{-}ER(C0_{direct}). \tag{13}$$

A value near zero indicates no material semantic penalty; a small positive value indicates a modest semantic cost; a negative value indicates that the masked workflow also improves semantic quality. In all cases, MQM is interpreted separately from structural failure rate.

6. Results

6.1. Experimental condition comparison: structural safety and semantic quality trade-off

The full outputs of the C0-C4 experimental conditions over the 3,000-segment subset are archived in the accompanying reproducibility artifact. Table 3 reports all five experimental conditions. In the table, MQM-ER is reported as mean ± approximate 95% half-width over repeated MQM judge runs. `Rule QA` is the deterministic dashboard score defined above, not a linguistic translation-quality score.

Table 3. *Comparison summary on the 3,000-segment subset.*

Backend	Cond.	Short name	Structure failure % ↓	Rule QA ↑	MQM-ER ↓
Grok 4.3	C0	Direct	5.67	93.60	6.289 ± 0.368
Grok 4.3	C1	Complete workflow	0.00	92.57	9.005 ± 2.807
Grok 4.3	C2	Direct + glossary	5.13	94.54	3.630 ± 1.488
Grok 4.3	C3	Masked without glossary	0.00	92.12	13.540 ± 4.216
Grok 4.3	C4	Single candidate	0.00	92.40	11.016 ± 2.372

Gemini 3.1 Flash-Lite	C0	Direct	8.70	92.72	7.194 ± 1.702
Gemini 3.1 Flash-Lite	C1	Complete workflow	0.00	92.97	11.109 ± 2.481
Gemini 3.1 Flash-Lite	C2	Direct + glossary	9.30	92.81	4.904 ± 1.945
Gemini 3.1 Flash-Lite	C3	Masked without glossary	0.00	92.88	13.500 ± 2.036
Gemini 3.1 Flash-Lite	C4	Single candidate	0.00	92.82	12.482 ± 3.276
TAIDE 12B	C0	Direct	30.10	76.11	37.339 ± 10.414
TAIDE 12B	C1	Complete workflow	0.00	84.52	40.736 ± 6.212
TAIDE 12B	C2	Direct + glossary	25.17	79.63	31.564 ± 5.943
TAIDE 12B	C3	Masked without glossary	0.00	81.85	47.200 ± 10.462
TAIDE 12B	C4	Single candidate	0.00	80.72	41.402 ± 6.335

Under the implemented conditions without masking, C0 and C2 retain non-zero structural risk for every backend: Grok C0 and C2 have structure-failure rates of 5.67% and 5.13%, Gemini C0 and C2 have rates of 8.70% and 9.30%, and TAIDE C0 and C2 have rates of 30.10% and 25.17%. These results do not claim to rule out all possible prompt-only methods; they show that direct generation leaves a residual token-preservation risk in this workflow. In contrast, all masked condition bundles produce final artifacts with zero observed structure failures under the token classes we have checked; candidates that fail strict validation are replaced by source-preserving fallbacks and flagged for review. The full item-level breakdown shows that direct generation failures concentrate in newline preservation, Qt shortcut-marker preservation, HTML/XML tag preservation, and placeholder preservation. Detailed counts are reported in the accompanying reproducibility artifact so as to keep this paper concise.

MQM shows the expected semantic trade-off. C2, the direct condition with glossary hints, obtains the lowest MQM error rate for all three backends, but it also retains non-zero structural failure rates. C1 has a higher MQM error rate than C0 and C2, while its masking, reassembly, validation, and fallback bundle produces structurally valid final artifacts under the extractors we implement. Thus, the direct conditions receive lower aggregate MQM-ER, whereas the selected production bundle gives stronger final-artifact structural protection.

Compared with C3, C1 has higher Rule QA and lower mean MQM-ER for all three backends. These results suggest a favorable effect of glossary hints in this experiment, but the differences are descriptive and no statistical significance is claimed.

6.2. The effect of generating multiple candidates

C1 and C4 use different limits for the number of generated candidates. Both use masking and glossary hints from tabular resources; C1 uses three candidates and C4 uses one. Both achieve zero observed structure failures, so candidate count is not the source of structural safety. Instead, multiple candidates mainly improve robustness by providing more candidate translations to choose from.

For cloud backends, the effect is small. Relative to C4, C1 improves Rule QA by +0.17 for Grok and +0.15 for Gemini. For TAIDE, the effect is larger: Rule QA improves by +3.80, and the

average valid candidate count increases from 0.689 to 2.119. MQM also favors C1 over C4 for all three backends, but the differences are modest relative to repeated-judge variability. We therefore interpret top-three candidate selection as a robustness mechanism rather than the primary safety contribution. Final-artifact structural protection comes from the combined masking, rule-based reassembly, validation, rejection, and fallback policy.

6.3. Complete-corpus C1 production results

Complete-corpus C1 experiments were run on all 28,924 QGIS source messages for each backend. We select C1 for our production workflow: masking, glossary hints from tabular resources, and top-three candidates.

Structure score is the arithmetic mean of eight preservation scores: Qt placeholders, brace placeholders, printf placeholders, HTML/XML entities, HTML/XML tags, numbers, newlines, and accelerators. For each class r, its score is $100(1 - n_r/N)$, where n_r is the number of segments with an observed preservation failure for class r and N is the number of messages checked. Rule QA is the deterministic dashboard score defined above. Possibly untranslated counts segments flagged by the rule-based detector as retaining substantial source-language residue.

Table 4. *Comparing the three LLMs under the production condition C1. Zero observed structural failure does not imply complete localization.*

Backend	Messages checked	Structure failure % ↓	Structure score ↑	Rule QA ↑	Avg. valid candidates	Possibly untranslated
Grok 4.3	28,924	0.00	100.000	92.11	2.938	1,394 (4.82%)
Gemini 3.1 Flash-Lite	28,924	0.00	100.000	90.09	2.935	1,825 (6.31%)
TAIDE 12B	28,924	0.00	100.000	65.13	2.052	6,674 (23.07%)

The zero rates in Table 4 describe the final `.ts` artifacts and include source-preserving fallbacks flagged for review: 591 of 28,924 rows for Grok (2.04%), 1,396 for Gemini (4.83%), and 7,265 for TAIDE (25.12%). These rows preserve structure but remain incomplete localization rather than accepted translated content.

The complete-corpus result confirms that the subset finding scales up to the complete QGIS Qt source file: all three backends produce final artifacts with zero observed structural failures under C1 for the token classes we have checked. The local TAIDE backend has zero observed structural failures under this workflow bundle, but its Rule QA is much lower. This indicates that the local backend is a viable offline path for structural preservation while requiring substantially more review effort than the two cloud backends.

7. Reproducibility and Artifact Availability

The qgis-llm-localization-workflow repository provides the workflow code, configurations, reports, and a compact 100-segment reproduction fixture. The complete archived outputs used for the paper tables—C0-C4 for the 3,000-segment subset and C1 for all 28,924 messages, across all three backends—are available in Depositar, identified by the persistent identifier `ark:37281/k553s150r`.

Table 5. *Reproducibility levels.*

Level	Command or location	New model calls?	What it verifies

Mini rule-based evaluation	`python scripts/run_repro.py full-mini; experiments/demo_ablation_grok_100/`	No	Reruns rule-based evaluation, scoring, condition comparison, and MQM request generation on bundled 100-segment C0-C4 `.ts` outputs.
Extended archived-output scoring	`python code/public_repository/scripts/run_repro.py score --experiment experiments/<experiment> --force-eval`	No	From the Depositar archive root, recomputes structural and deterministic Rule QA metrics from archived generated `.ts` files rather than from preformatted table rows.
Optional MQM or translation rerun	`python scripts/run_repro.py mqm ... --run-grok; python scripts/run_repro.py translate ...`	Yes	Rebuilds MQM judge outputs or regenerates a small translation test run using cloud AI credentials or local inference.

The default mini rule-based evaluation is reviewer-friendly and can be used offline. It uses the same C0-C4 condition definitions and the same rule-based evaluator/scorer as this paper's experiments, but on a smaller 100-segment archived-output fixture so that it can run quickly. It should be interpreted as an executable validation of the rule-based evaluation, scoring, comparison, and request-planning code paths, not as a full regeneration of LLM translation outputs or as a substitute for the 3,000-segment and 28,924-message experimental artifacts.

The compact repository release contains the source `.ts` file, tabular glossary resources, C0-C4 configuration files, workflow scripts, the 100-segment fixture and demo outputs, workflow manifests, rule-based evaluation outputs, MQM request-planning outputs, and an `.env.example` file. The Depositar archive contains the generated `.ts` outputs and evaluation reports for the reported 3,000-segment C0-C4 comparison and 28,924-message C1 runs. With that archive, the same scoring command recomputes rule-based metrics from archived outputs without new model calls. Rerunning translation or MQM judging requires the appropriate cloud LLM API credentials or local inference hardware. API keys are excluded from the repository and are supplied through environment variables such as `XAI_API_KEY` and `GEMINI_API_KEY`.

In addition to the reproducibility archive, the released Traditional Chinese localization files are deposited as public datasets in Depositar, a public repository for research datasets, with persistent ARK identifiers. The QGIS 4.0 and QGIS 3.44 zh-Hant localization datasets were generated with the assistance of GPT-4.1-mini and released in March 2026, before the workflow presented in this paper was formalized, although they were based on the same general idea of masked template translation with glossary hints. The released datasets include the QGIS 4.0 zh-Hant Translation Dataset (TS/QM Files), identified by `ark:37281/k5f167n46` and available at https://pid.depositar.io/ark:37281/k5f167n46, and the Long Term Release QGIS 3.44 zh-Hant Translation Dataset (TS/QM Files), identified by `ark:37281/k5g11462n` and available at https://pid.depositar.io/ark:37281/k5g11462n. These deposits provide persistent access to the released translation files; the GitHub repository provides the workflow and 100-segment fixture, while the paper's complete experimental outputs are available in the Depositar archive (`ark:37281/k553s150r`).

The code repository README documents the quickstart, rule-based scoring, MQM request planning, optional MQM judge run, and optional API translation test run.

8. Discussion and Limitations

Our proposed workflow improves reliability by separating structural safety, terminology guidance, semantic quality evaluation, and human review. The rule-based layer is the most objective because it checks formatting artifacts that should be preserved. The structure-preserving template layer strengthens this idea by removing protected tokens from the LLM generation space. Generating multiple candidates is a secondary robustness layer; it improves candidate availability, especially for the local backend. Structural protection in the final artifacts comes from the complete masking, reassembly, validation, rejection, and fallback policy.

The role of masking should be interpreted carefully. Masking does not solve semantic adequacy by itself, and the MQM results show a modest semantic cost for C1 compared with C0 and C2, the two conditions without using masking. However, this is a different risk category from interface breakage. C0 and C2 provide evidence of residual risk under the implemented direct generation settings, not a complete dismissal of all prompt-only approaches. C2 can be semantically stronger, but it still leaves non-zero structural failure rates in this experiment. In practical terms, the selected production bundle trades a measured semantic cost and a review queue for stronger final-artifact structural reliability under the token classes we have checked.

The local LLM results should be read as evidence for deployability, not model superiority. TAIDE achieves the same zero observed structural failure result under C1, but its Rule QA and review-workload indicators are much worse than the cloud backends. In the archived status diagnostics, 7,265 of 28,924 TAIDE C1 rows, or 25.12%, entered a review-required fallback state. The external run log makes these rows identifiable even though the generated `.ts` file preserves their source text. Using the local backend therefore requires substantially more human review before release; in the present configuration, TAIDE should be treated as an offline deployment path with a larger review queue rather than as a drop-in substitute for the cloud backends.

The glossary layer depends on glossary quality. Because glossary matches are hints rather than replacements, the workflow avoids many unnatural literal substitutions, but terminology correctness still requires evaluation.

Semantic quality review is necessary because the rule-based layer does not evaluate adequacy, fluency, domain correctness, or target-locale style. LLM judges can be useful for large-scale review, but their judgments are assisted annotations rather than ground truth. Grok 4.3 is the sole MQM judge here and evaluates outputs from every backend, including Grok itself; self-preference and model-family bias therefore cannot be excluded. Human review remains necessary for release-critical strings, ambiguous GIS terms, and user-facing messages with high impact.

8.1. Practical deployment recommendation

For a FOSS localization team, the recommended deployment workflow is:

1. Run the selected production condition.
2. Keep the external run log with the generated `.ts` artifact so fallback rows remain identifiable.
3. Accept only rows with zero blocking structural issues.

4. Treat review-required rows as incomplete, not as translated content.
5. Prioritize review by English residue, terminology warnings, semantic severity, and review status.
6. Run XML well-formedness and rule-based validation before each release candidate.
7. Use human review for final acceptance of UI-critical strings and domain terminology.

Such a workflow preserves the format integrity of the translated messages and creates auditable review queues; it does not eliminate all localization errors or claim superior translation quality.

9. Conclusion

This paper presents a Python workflow for reliable localization of QGIS Qt `.ts` files with LLM support, without tying the method to a specific LLM backend. The workflow supports both cloud-based and local LLMs, uses tabular glossary resources as contextual terminology hints, and applies structure-preserving template translation to preserve protected tokens through rule-based reassembly.

The 3,000-segment subset comparison shows that the implemented conditions without masking produce non-zero structural failure rates for Grok, Gemini, and TAIDE, while the masked condition bundles produce final artifacts with zero observed structure failures under checked token classes for all three backends. Complete-corpus C1 results extend this finding to all 28,924 QGIS source messages; this zero-failure result includes source-preserving fallback rows that remain incomplete and require review. Semantic quality results using MQM show that masking is not a semantic quality optimizer: C1 has a modestly higher MQM error rate than the direct baselines, and C2 has the lowest MQM-ER but retains non-zero observed structural failure rates. C1 versus C4 further shows that generating three candidates helps robustness, especially for TAIDE, while final-artifact structural protection comes from the combined masking, rule-based reassembly, validation, rejection, and fallback policy.

The broader contribution of this research is methodological. Reliable FOSS localization should not depend on raw LLM output alone. It should be built as a reproducible workflow in which rule-based validation protects software artifact structure, glossary retrieval supports terminology consistency, MQM evaluation with LLM assistance scales up semantic quality review, and human reviewers retain final responsibility.

10. Acknowledgements and AI Disclosure

Generative AI was used for English wording, formulation, and formatting assistance. The authors verified the technical claims, references, experimental results, source code, and final manuscript text. LLM-generated translations are treated as drafts requiring validation and human review; MQM-style judgments are treated as assisted annotations rather than ground truth.

11. Funding

This research is supported in part by an Academia Sinica Thematic Project grant (no. AS-TP-114-L01; *Sound Atlas: Mapping the Changing Terrestrial, Marine, and Cultural Soundscapes for a Sustainable Island Social-Ecological System*) and a grant from the National

Science and Technology Council of Taiwan (no. NSTC 114-2621-M-001-001; *Advancing Research Data Infrastructures and Management Practices: Tools, Services, and Communities*).

References

[1] QGIS Development Team, "QGIS Geographic Information System," technical report, 2026. doi: 10.5281/zenodo.6139224.

[2] A. Graser, T. Sutton, and M. Bernasocchi, "The QGIS Project: Spatial without Compromise," *Patterns*, vol. 6, no. 7, p. 101265, 2025, doi: 10.1016/j.patter.2025.101265.

[3] The Qt Company, "TS File Format | Qt Linguist Manual." [Online]. Available: https://doc.qt.io/qt-6/linguist-ts-file-format.html

[4] B. Esselink, *A Practical Guide to Localization*. John Benjamins Publishing Company, 2000. doi: 10.1075/liwd.4.

[5] L. Bowker, *Computer-Aided Translation Technology: A Practical Introduction*. University of Ottawa Press, 2002. doi: 10.1353/book6554.

[6] Google AI for Developers, "Structured outputs | Gemini API." [Online]. Available: https://ai.google.dev/gemini-api/docs/structured-output

[7] xAI, "Structured Outputs | xAI Docs." [Online]. Available: https://docs.x.ai/developers/model-capabilities/text/structured-outputs

[8] Multidimensional Quality Metrics, "What Is MQM?." [Online]. Available: https://themqm.org/

[9] A. Lommel, H. Uszkoreit, and A. Burchardt, "Multidimensional Quality Metrics (MQM): A Framework for Declaring and Describing Translation Quality Metrics," *Tradumàtica: tecnologies de la traducció*, no. 12, pp. 455–463, 2014, doi: 10.5565/rev/tradumatica.77.

[10] T. Kocmi and C. Federmann, "GEMBA-MQM: Detecting Translation Quality Error Spans with GPT-4," in *Proceedings of the Eighth Conference on Machine Translation*, 2023, pp. 768–775. doi: 10.18653/v1/2023.wmt-1.64.

[11] Google AI for Developers, "Gemini 3.1 Flash-Lite." [Online]. Available: https://ai.google.dev/gemini-api/docs/models/gemini-3.1-flash-lite

[12] xAI, "Grok 4.3 | xAI Docs." [Online]. Available: https://docs.x.ai/developers/models/grok-4.3

[13] TAIDE Project, "taide/Gemma-3-TAIDE-12b-Chat-2602." [Online]. Available: https://huggingface.co/taide/Gemma-3-TAIDE-12b-Chat-2602

[14] Google, "google/gemma-3-12b-pt." [Online]. Available: https://huggingface.co/google/gemma-3-12b-pt

[15] Gemma Team, "Gemma 3 Technical Report." [Online]. Available: https://arxiv.org/abs/2503.19786

[16] J. A. Hanley and A. Lippman-Hand, "If nothing goes wrong, is everything all right? Interpreting zero numerators," *JAMA*, vol. 249, no. 13, pp. 1743–1745, 1983, doi: 10.1001/jama.1983.03330370053031.